

Version Control System

Research and Development Team

Nagarjuna College of Information Technology

Table of Contents

Abstract	3
Purpose	4
Background	5
Introduction	7
Version Control System	8
Centralized Version Control Systems (CVCS)	9
Distributed Version Control Systems (DVCS)	10
Application of VCHs	12
Software merging	12
Collaboration Modeling	12
Software Changes	13
Software branching	13
Open Source Software Projects	14
Advantages of VCS	15
Conclusion	16
Bibliography	17

Abstract

Version Control System(VCS) have been used by many developers during project development as it helps them to manage the source code and enables them to keep every version of the project they have worked on. It is the way towards managing, organizing and contributing the development of projects. In Software engineering, Software developers need to collaborate with each other to develop a better project. Thus VSC is very useful because it also supports a collaborative framework that makes it easy for software developer to work together effectively. Without VCS, collaboration is very challenging. This paper discusses the background related works about VCS that have been studied by researchers. The purpose of this paper is to convey the knowledge and ideas that have been established on VCS.

Purpose

The purpose of this research paper is to investigate and evaluate the significance and effectiveness of version control systems in the domain of software development. By examining the principles, features, and functionalities of various version control systems, this study aims to shed light on their impact on collaboration, code management, and project development.

Background

It all started with engineering

The engineering process involved identifying the assemblies and components in a design, and representing them in a hierarchical map. Each component was defined by its technical drawing, and the 'spec' which specified the materials and manufacturing techniques.

Having defined the context of a component within the overall design, for example the dimensions within which the component had to fit, and the tolerances within which it had to work, a team of engineers could work on each one relatively independently.

Over the course of the engineering process, they would construct and test the various assemblies, and change the drawings and specifications as required, giving each version of every component a version number.

Software soon followed

Version control for software mimicked the established techniques of the drawing office, but only the parts that were possible within the crude technology of the time. Back in the days of punch cards, once the programmers had finished with their routine, they would take the punched cards, change cards, or coding forms to the version-control or configuration-management team who would duly update the library, and build the customer-ready product from the official version.

At the same time, there was an office process that recorded the work and managed the versioning task. The work was treated as if it were a technical drawing. To alter a version, you checked the code in and out as though it were a technical drawing that had to be altered.

As soon as programs became text files, the manual process was automated using programs such as Source Code Control System (SCCS). At this stage, there wasn't the concept of storing just the difference between file versions. The technology was there but we weren't culturally ready.

This practice of delta-storage became mainstream in the 1980s with the Revision Control System (RCS), but the whole process still used the analogy of the file being the unit of work, as if it were a technical drawing.

The idea of being able to deal with a whole lot of files at once took a long time to catch on, as did having more than one person working on a file at the same time. Although Concurrent Versions Systems (CVS) was the first to break free from the idea of locking out the file to work on, it was unreliable until the introduction of Subversion.

Even now, it is only too easy for the version control system to fail to sort out the merging of two simultaneous alterations of a version file, and 'merge failure' is still a phrase dreaded by most developers.

Introduction

In software engineering, version control (also known as revision control, source control, or source code management) is a class of systems responsible for managing changes to computer programs, documents, large web sites, or other collections of information. Version control is a component of software configuration management.

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. For the examples in this book, you will use software source code as the files being version controlled, though in reality you can do this with nearly any type of file on a computer.

If you are a graphic or web designer and want to keep every version of an image or layout (which you would most certainly want to), a Version Control System (VCS) is a very wise thing to use. It allows you to revert selected files back to a previous state, revert the entire project back to a previous state, compare changes over time, see who last modified something that might be causing a problem, who introduced an issue and when, and more. Using a VCS also generally means that if you screw things up or lose files, you can easily recover. In addition, you get all this for very little overhead.

Version Control System

According to Otte, there are two different approaches to VCSs, which are the Centralized Version Control System (CVCS) and the Distributed Version Control System (DVCS). CVCS is a centralized model with one central repository while DVCS is a distributed model without a central repository but has a local repository for every user. The most commonly used CVCS tools are CVS and Subversion. Since the emergence of DVCS tools such as **Git**, **Mercurial**, **Bazaar**, and **BitKeeper**, many open and closed source projects have been proposing to move or have already moved their source code repositories to a DVCS . Summary of the comparison between CVCS and DVCS is shown in the Table 1 below.

Version Control System	CVCS	DVCS
Repository	There is only one central repository which is the server.	Every user has a complete repository which is called local repository on their local computer.
Repository Access	Every user who needs to access the repository must be connected via network.	DVCS allows every user to work completely offline. But user need a network to share their repositories with other users.
Example of VCS Tools	Subversion, Perforce Revision Control System	Git, Mercurial, Bazaar, BitKeeper
Software Characteristics that suitable	i. Projects that allow only several users to contribute to the software development. ii. Team located in a single site.	i. DVCS is suitable for a single or more developers because the project repository is distributed to all the developers and this ability offer a great improvement for the projects. ii. It also can be applied for a small or big software projects because it makes less difficult for normal users to contribute to the development. iii. Team located in multiple site or different countries and different timezones.

Centralized Version Control Systems (CVCS)

CVCS is an approach that enables the developers to work cooperatively. It stores a master copy of files history and in order to read, retrieve, commit new changes to a certain version, they need to contact a server.

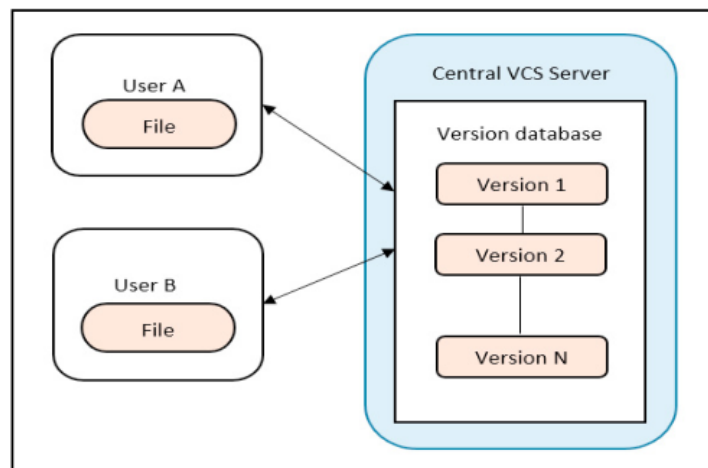


Fig. 1 Somasindaram R. (2013). Centralized version control [Digital image]. Retrieved April, 2018.

As shown in Fig. 1, CVCS has a single repository which is basically the server. All the files were kept in the server that everybody has access to from their local computer. All the developers make changes against this repository through a checkout taken from the repository but only the last version of the files are retrieved. As the files are stored in the server, it means that any changes made will automatically be shared with other developers.

The developers otherwise known as the contributors and committers, must be the core developers who perform basic tasks such as creating branches, merging branches or reverting changes to previous states 10. In spite of that, this restriction will affect the participation and authorship of new contributors because in order to perform basic tasks, they need to follow some steps in joining the process to become a core developer.

There is potential risk involved in using a CVCS because the users only have the last version of the files in their system for working purposes. Most commonly used CVCS are Concurrent Version System (CVS), Subversion (SVN), and Perforce.

Distributed Version Control Systems (DVCS)

Nowadays, Open Source Software (OSS) projects largely adopt the DVCS. This is because of the risk of using the Local Version Control and CVCS. When a project is using a CVCS, it means that the history related to the projects will be kept in a single place, while developers will lose the power to work collaboratively when using the local version control. If a developers are using CVCS, it will works well if they can contact the server but they cannot contact the server, it will be a big problem for them. The risks stated are the reasons why most OSS projects are adopting the new version control system for recording, managing, and propagating their source code changes. DVCS is a combination of advantages of both concepts and a hybrid system.

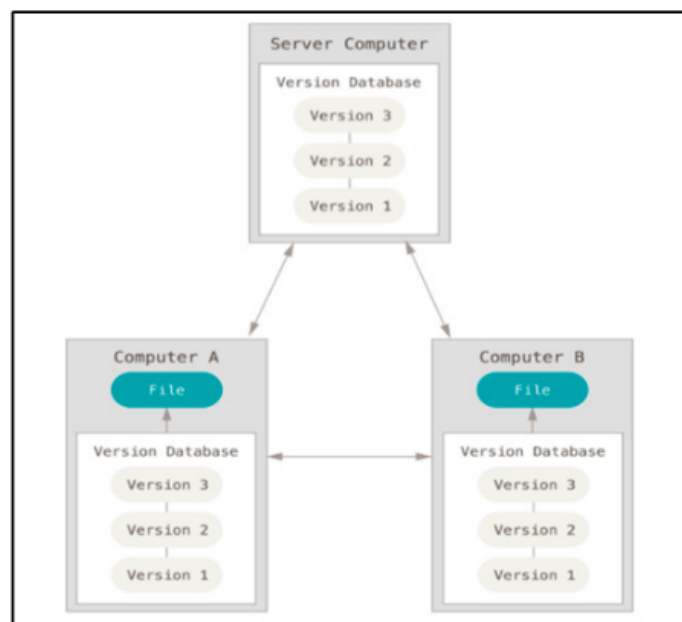


Fig. 2 Chacon S & Strauv B (2013). Distributed version control system [Digital image]. Retrieved April, 2018.

As shown in Fig. 2, fundamentally, DVCS is designed to act in both ways as it stores the entire history of the files on each machine locally²¹ and it can also sync the local changes made by the user back to the server whenever required, so that the changes can be shared with the whole team ¹¹. Clients can communicate with each other and maintain their own local branches without having to go through a central server or repository. Then, synchronization takes place between the peers who decide which change sets to exchange. Clients do not just look into the latest snapshot of the files but they are able to totally copy the repository. Therefore, if any server breaks down, and these systems were collaborating using it, any of the client repositories can be mirrored back to the server to recover it.

Every clone is genuinely a full backup of all the data. Examples of DVCS tools are **Git**, **Bazaar**, **Mercurial**, **Bitkeeper**, and **Darcs**. Bitkeeper and Bazaar are two of the earliest DVCS tools. **Git** and **Mercurial** are the newest DVCS tools that improve the merging and branching features.

Application of VCHs

Numerous researches have focused on version control systems. Subsequently, we grouped the related works that have been studied by researchers according to the application of VCS based on our analysis and discussed in the following subtopics.

Software merging

Conflict resolution strategies adopted by version control systems can be classified into three which are *unstructured*, *structured*, and *semi-structured*. Semi-structured approach is the combination of the strength of both, the expressiveness of the structured merge, and the generality of the unstructured merge. The purpose of the semi-structured approach is to be responsible for providing the information regarding the software artifacts so that the conflicts that occur can be resolved automatically. 154 of merge scenarios from 10 software projects that used *Git* as the VCS were used to conduct the study. It was found that the semi-structured approach could extensively reduce the number of conflicts that occurred and it was able to resolve the ordering conflicts that rely on the software's elements order.

Collaboration Modeling

Alam et al. provided a thorough analysis in an article entitled Towards Collaborative Modelling Using a Concern-Driven Version Control System. In the paper, the authors stated that the *Concern-Oriented Reuse* (CORE) approach is the idea for a collaboration in modeling. According to the authors, CORE is a novel reuse paradigm that assists broad-based model reuse. Software developers widely use VCS in order to team up and track development activities. The reuse in software development can be categorized by class libraries, services, and components facilitated by collaboration and coordination during the development activities. As a result of the effectiveness of the reuse in software development, software developers can reuse the artifacts developed by other developers. The advancement in reuse in VCS allows software developers to be more collaborative. This approach will make collaboration in modeling easier, simpler, and faster with modular ways to settle conflicts that occur.

Debrececi et al. explained that the MONDO COLLABORATION FRAMEWORK provides a rule-based fine-grained model-level secure access control, property-based locking, and automated model merge integrated over existing VCS such as Subversion and Git for storage and version control. Large projects worked by multiple developers are generally not feasible for the system to be divided into isolated modules for each developer. In order to extend the traditional VCS, the MONDO COLLABORATION FRAMEWORK advances secure collaborative modeling features as follows:

- Integration with VCS;
- Fine-grained model-level access control;
- Property-based locking;
- Automated model merge; and
- Offline and online collaboration.

Software Changes

Brindescu et al. provided a thorough analysis in an article entitled How Do Centralized and Distributed Version Control Systems Impact Software Changes . In the article, the authors stated that developers have little knowledge of whether they are benefitting from the use of DVCS. The purpose of this study is to look at the influence of DVCS in terms of splitting, grouping, and committing changes. 820 participants were recruited for a survey and 409m lines of changed codes were analyzed in 358300 commits. It was found that different approaches of VCS have different effects on developers, teams, and processes.

Software branching

Another article considered was written by **Barr et al.**, *Cohesive and Isolated Development* with Branches, that discussed how the DVCS branching process allows developers to collaborate on tasks in highly cohesive branches while enjoying reduced interference from any developers working on other tasks even though the tasks are strongly coupled to theirs. According to the authors, Open Source Software (OSS) projects are rapidly adopting DVCS because their project histories are easier to understand when faced with maintenance tasks and they are easier to revert to a previous state if the branch created has a problem. In

order to understand how DVCS branches protect developers from interruptions, an evaluation was done on how branches are used and the benefit that could be gained by examining Linux history.

Open Source Software Projects

Another article that was studied is from Rodriguez-Bustos and Aponte, How Distributed Version Control Systems Impact Open Source Software Projects 10. DVCS has technical facilities or features that assist software developers to work in new ways. In this paper, a preliminary study was conducted to evaluate the impact of the migration process on a developer's organization and its contribution by analyzing Mozilla repositories that were migrated from CVS to Mercurial in 2007.

Advantages of VCS

This section gives an overview of the advantages that VCS offers. VCS has been proven to accelerate and simplify the software development process. There are loads of advantages in using VCS for software projects. VCS allows people to work absolutely freely with the team. They can work on any file at any time without overlapping each other's work by writing over other people's code. If two developers make changes in the same file, the VCS will merge the changes or warn them that some of the codes are conflicting because VCS has the ability to track each alteration or changes made in the files or codes. The latest or the whole project will be in the VCS. Obviously, since VCS enables collaboration so people can share data, files, and documents more easily through the use of VCS.

Saving a version of the project after making changes is an important task. This task becomes tedious and quite confusing if people are not using VCS. Every time people commit changes, they create a new version of the corresponding file. VCS saves the version in a professional way. It does not need an extra folder or a complete copy of the project. When saving a new version, it also saves when those changes were made and by whom and also a message that describes the changes, so that the team will understand them later. When a certain version of the project is needed, software developers can request it at any time and they will have a snapshot of the complete project.

Conclusion

Software developers should have a rudimentary understanding of what VCS is and which type of VCS suits them. The adoption of a VCS is a must in software development. It helps software developers manage their codes easily because it is common to have a lot of changes involving addition or deletion of features. In order to adopt a VCS, a software developer must know and perfectly understand which approach should be used as it will affect the whole project and team. It is also important for them to have the knowledge of different approaches of VCS because the various approaches will affect their software development process differently.

Bibliography

1. *Version control*. (n.d.). Wikipedia. Retrieved June 27, 2023, from https://en.wikipedia.org/wiki/Version_control
2. Getting Started - About Version Control." n.d. Git SCM. Accessed June 27, 2023. <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>.
3. Zolkipli, Nazatul N., Amir Ngah, and Aziz Deraman. 2018. "Version Control System: A Review." *Version Control System: A Review*, (January), 9. 10.1016/j.procs.2018.08.191.
4. Carstensen, Wibke. "A brief history of version control | Redgate." Redgate Software, 7 November 2016, <https://www.red-gate.com/blog/database-devops/history-of-version-control>. Accessed 28 June 2023.